AN OPEN-ENDED PROJECT REPORT

on

Smart Navigation System

As a part of

Data Structure and Algorithms (CE0417)

Submitted by

Jenish Pancholi (IU2441230510)

Nandan Khagram (IU2441230559)

Jainesh Patel (IU2441230479)

In fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE ENGINEERING

INSTITUTE OF TECHNOLOGY AND ENGINEERING

INDUS UNIVERSITY CAMPUS, RANCHARDA, VIA-THALTEJ

AHMEDABAD-382115, GUJARAT, INDIA,

WEB: www.indusuni.ac.in

APRIL 2026

1. OPEN-ENDED PROJECT TITLE:

Smart Navigation System

2. Concepts used:

> Data Structures:

1. Graph (Adjacency list)

vector<vector<edge>>

2. Vectors

Used as a dynamic array

3. Stack

In Djikstra's algorithm

> Algorithms:

1. Djikstra's Algorithm:

Used to predict the shortest path. Djikstra's algorithm is used here as it is better than bfs since that does not consider weight of the edges.

2. Linear Regression (2D):

Used to get a general idea of the time. Each node constitutes of traffic and distance with are used to train for time.

3. Project Description:

- This project implements a Smart Traffic Routing System using a weighted directed graph where nodes represent locations and edges represent roads with distance and traffic conditions. It predicts travel time using a simple linear regression model and dynamically updates edge weights to reflect real-time traffic influence for efficient path computation.
- linear regression is trained on distance and traffic data to estimate travel time. These predicted weights are then used in Dijkstra's algorithm to find the shortest-time path between source and destination, making routing more adaptive compared to static shortest-path approaches.
- It demonstrates a hybrid approach combining **data-driven prediction and classical graph theory**. The model learns from historical road data to estimate

travel cost, while Dijkstra's algorithm computes the optimal route. This makes the system suitable for intelligent navigation applications where traffic conditions significantly affect travel efficiency.

- Linear regression uses the formula y = mx1 + nx2 + b, or in other words, time = w1*distance + w2*traffic + b

Graph Representation:

0 -> 1 (5, 0.2)

0 -> 2 (8, 0.7)

1 -> 3 (4, 0.3)

1 -> 4 (6, 0.8)

1 -> 2 (2, 0.1)

2 -> 3 (3, 0.5)

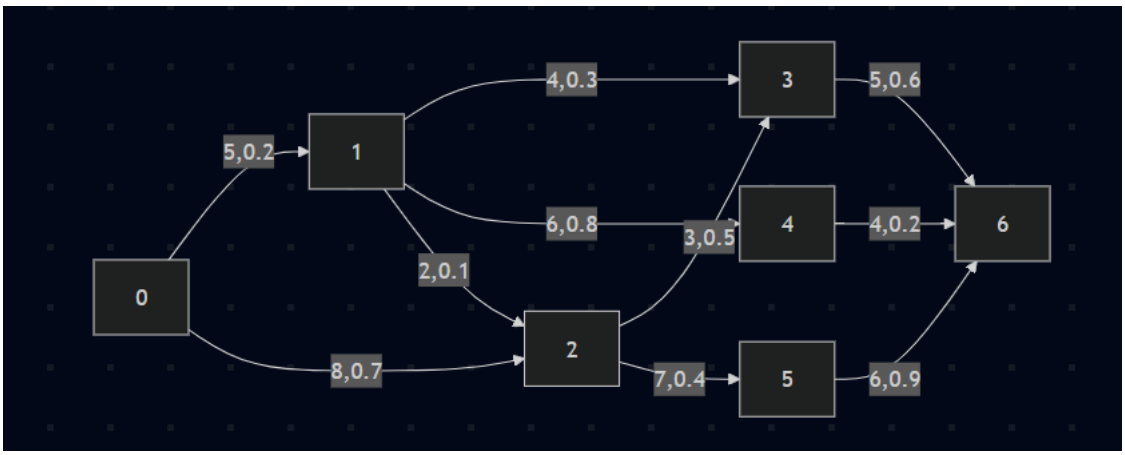
2 -> 5 (7, 0.4)

3 -> 6 (5, 0.6)

4 -> 6 (4, 0.2)

5 -> 6 (6, 0.9)

Graph Visualisation:



4. Code:

```
project.cpp > ...
#include<iostream>
#include<vector>
#include<stack>
using namespace std;

class Edge{
public:
    int dest;
    double dist;
    double traf;
    double predWt;

    Edge(int dest,double dist,double traf){
        this->dest=dest;
        this->dist=dist;
        this->traf=traf;
        predWt=0.0;
    }
};

class Graph{
public:
    int numNodes;
    vector<vector<Edge>>adjList;

    void init(int n){
        numNodes=n;
        adjList.resize(n);
    }

    void addEdge(int u,int v,double dist,double traf){
        Edge e(v, dist, traf);
        adjList[u].push_back(e);
    }

    void displayGraph(){
        cout<<"Initail Graph\n";
        cout<<"From      To       Distance        Traffic\n";
        for(int u=0;u<numNodes;u++){
            for(int i=0;i<adjList[u].size();i++){
                Edge &e=adjList[u][i];
                cout<<u<<"          "<<e.dest<<"        "<<e.dist<<"         "<<e.traf<<"\n";
            }
```

```
        }
        cout<<endl;
    }

    void displayUpdatedGraph(){
        cout<<"Updated Graph:\n";
        cout<<"From  To  Distance  Traffic  PredictedTime\n";
        for(int u=0;u<numNodes;u++){
            for(int i=0;i<adjList[u].size();i++){
                Edge &e=adjList[u][i];
                cout<<u<<"     "<<e.dest<<"   "<<e.dist<<"       "<<e.traf<<"    "<<e.predWt<<endl;
            }
        }
    }
};

class LinReg{
public:
    double w1,w2,b;
    double alpha;
    int iter;

    void init(double a,int it){
        w1=0.1;
        w2=0.1;
        b=0.0;
        alpha=a;
        iter=it;
    }

    double computeLoss(vector<double>d,vector<double>t,vector<double>y){
        double loss=0.0;
        int n=d.size();
        for(int i=0;i<n;i++){
            double pred=w1*d[i]+w2*t[i]+b;
            double err=pred-y[i];
            loss=loss+(err*err);
        }
        return loss/n;
    }
```

```
    void train(vector<double>d,vector<double>t,vector<double>y){
        int n=d.size();
        for(int it=0;it<iter;it++){
            double dw1=0,dw2=0,db=0;
            for(int i=0;i<n;i++){
                double pred=w1*d[i]+w2*t[i]+b;
                double error=pred-y[i];
                dw1+=error*d[i];
                dw2+=error*t[i];
                db+=error;
            }
            w1=w1-(alpha*(2.0/n)*dw1);
            w2=w2-(alpha*(2.0/n)*dw2);
            b=b-(alpha*(2.0/n)*db);

            if(it % 3000 == 0){
                cout<<"Iteration "<<it<<" | w1: "<<w1<<" w2: "<<w2<<" b: "<<b<<endl;
            }
        }
    }

    double predict(double dist,double traf){
        return w1*dist+w2*traf+b;
    }
};

vector<int> dijkstra(Graph &g,int source,int dest){
    int n=g.numNodes;
    vector<double> dist(n,1e9);
    vector<bool> visited(n,false);
    vector<int> parent(n,-1);
    dist[source]=0;

    for(int count=0;count<n-1;count++){
        int u=-1;
        double minDist=1e9;

        for(int i=0;i<n;i++){
            if(!visited[i]&&dist[i]<minDist){
                minDist=dist[i];
                u=i;
            }
```

```
        if(u==-1)break;

        visited[u]=true;

        for(int i=0;i<g.adjList[u].size();i++){
            Edge &e=g.adjList[u][i];
            int v=e.dest;
            double weight=e.predWt;

            if(!visited[v]&&dist[u]+weight<dist[v]){
                dist[v]=dist[u]+weight;
                parent[v]=u;
            }
        }
    }

    vector<int> path;
    if(dist[dest]==1e9){
        return path;
    }

    stack<int> st;
    int x=dest;
    while(x!=-1){
        st.push(x);
        x=parent[x];
    }

    while(!st.empty()){
        path.push_back(st.top());
        st.pop();
    }

    cout<<"Shortest Time: "<<dist[dest]<<endl;
    cout<<"Dijkstra Path: ";
    for(int i=0;i<path.size();i++){
        cout<<path[i];
        if(i+1<path.size())cout<<" -> ";
    }
    cout<<endl;

```

```
    return path;
}

void reconstructPath(Graph &g,vector<int>path){
    if(path.empty()){
        cout<<"No path found\n";
        return;
    }

    double totalDist=0,totalTime=0;
    cout<<"From To Dist Time\n";

    for(int i=0;i+1<path.size();i++){
        int u=path[i],v=path[i+1];

        for(int j=0;j<g.adjList[u].size();j++){
            if(g.adjList[u][j].dest==v){
                Edge &e=g.adjList[u][j];
                cout<<u<<"   "<<v<<"   "<<e.dist<<"   "<<e.predWt<<"\n";
                totalDist+=e.dist;
                totalTime+=e.predWt;
                break;
            }
        }
    }

    cout<<"\nFinal Route: ";
    for(int i=0;i<path.size();i++){
        cout<<path[i];
        if(i+1<path.size())cout<<" -> ";
    }

    cout<<"\nTotal Distance: "<<totalDist;
    cout<<"\nTotal Time: "<<totalTime;
}

int main(){
    cout<<"SMART TRAFFIC SYSTEM\n";

    Graph g;
    g.init(7);

    g.addEdge(0,1,5,0.2);
```

```
    g.addEdge(0,1,5,0.2);
    g.addEdge(0,2,8,0.7);
    g.addEdge(1,3,4,0.3);
    g.addEdge(1,4,6,0.8);
    g.addEdge(1,2,2,0.1);
    g.addEdge(2,3,3,0.5);
    g.addEdge(2,5,7,0.4);
    g.addEdge(3,6,5,0.6);
    g.addEdge(4,6,4,0.2);
    g.addEdge(5,6,6,0.9);

    g.displayGraph();

    vector<double>d={5,8,4,6,3,7,5,4,6,2};
    vector<double>t={0.2,0.7,0.3,0.8,0.5,0.4,0.6,0.2,0.9,0.1};
    vector<double>y={7,18,7.5,20,8,14,13.5,6.5,22,4};

    LinReg lr;
    lr.init(0.001,30000);
    lr.train(d,t,y);

    for(int u=0;u<g.numNodes;u++){
        for(int i=0;i<g.adjList[u].size();i++){
            double p = lr.predict(g.adjList[u][i].dist, g.adjList[u][i].traf);
            if(p<1){
                p=1;
            }
            g.adjList[u][i].predWt=p;
        }
    }

    g.displayUpdatedGraph();

    vector<int> path=dijkstra(g,0,6);
    reconstructPath(g,path);

    return 0;
}
```

OUTPUT:

```
PS E:\Indus Project\DSA> g++ project.cpp
PS E:\Indus Project\DSA> ./a.exe
SMART TRAFFIC SYSTEM
Initail Graph
From        To        Distance        Traffic
0           1         5          0.2
0           2         8          0.7
1           3         4          0.3
1           4         6          0.8
1           2         2          0.1
2           3         3          0.5
2           5         7          0.4
3           6         5          0.6
4           6         4          0.2
5           6         6          0.9

Iteration 0 | w1: 0.230774 w2: 0.113706 b: 0.023006
Iteration 3000 | w1: 2.17166 w2: 4.02464 b: -0.582212
Iteration 6000 | w1: 1.99384 w2: 6.91138 b: -1.10514
Iteration 9000 | w1: 1.83214 w2: 9.15757 b: -1.38184
Iteration 12000 | w1: 1.69374 w2: 10.9061 b: -1.52733
Iteration 15000 | w1: 1.57926 w2: 12.2675 b: -1.60309
Iteration 18000 | w1: 1.48649 w2: 13.3278 b: -1.64196
Iteration 21000 | w1: 1.4123 w2: 14.1536 b: -1.66144
Iteration 24000 | w1: 1.35347 w2: 14.7969 b: -1.67082
Iteration 27000 | w1: 1.30708 w2: 15.298 b: -1.67502
Updated Graph:
From  To  Distance  Traffic  PredictedTime
0     1   5       0.2    7.81431
0     2   8       0.7    19.4704
1     3   4       0.3    8.11248
1     4   6       0.8    18.4979
1     2   2       0.1    2.43352
2     3   3       0.5    9.97947
2     5   7       0.4    13.4933
3     6   5       0.6    14.0896
4     6   4       0.2    6.54366
5     6   6       0.9    20.0667
Shortest Time: 30.0164
Dijkstra Path: 0 -> 1 -> 3 -> 6
From To Dist Time
0   1   5   7.81431
1   3   4   8.11248
3   6   5   14.0896

Final Route: 0 -> 1 -> 3 -> 6
Total Distance: 14
Total Time: 30.0164
PS E:\Indus Project\DSA>
```